VOLUME 03 OF 10

C++, Algorithms and Language Implementation

Undergraduate CS core

Write native software safely, select data structures, analyze algorithms and build a tiny language.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 11 Native programming with C++

11.1 Compilation, linking and build systems • 11.2 Integer, floating-point and object representations • 11.3 Values, references, pointers and lifetimes • 11.4 Standard containers, iterators and error handling

Module 12 Ownership, abstractions and C++ design

12.1 RAII and resource ownership • 12.2 Copy, move and exception safety • 12.3 Interfaces, composition and polymorphism • 12.4 Templates, concepts and data-oriented choices

Module 13 Data structures and their costs

13.1 Complexity, arrays and linked structures • 13.2 Stacks, queues and ring buffers • 13.3 Hash tables and collision policies • 13.4 Trees, heaps and priority queues

Module 14 Algorithms for maps, simulation and tools

14.1 Searching, sorting and algorithm selection • 14.2 Graph traversal and dependency ordering • 14.3 Dijkstra, A* and admissible heuristics • 14.4 Dynamic programming, complexity limits and approximation

Module 15 Programming languages, compilers and computability

15.1 Language paradigms and type systems • 15.2 Lexing, parsing and syntax trees • 15.3 Bytecode, virtual machines and bounded execution • 15.4 Automata, computability and compiler optimization

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 11

Native programming with C++

Prerequisites Volume 01, Modules 06-07; do not skip the ownership module that follows.

Four lessons

11.1 Compilation, linking and build systems

Explain how source code becomes a runnable native program.

11.2 Integer, floating-point and object representations

Choose native types with explicit range and precision assumptions.

11.3 Values, references, pointers and lifetimes

Track where an object lives and how long access remains valid.

11.4 Standard containers, iterators and error handling

Use library abstractions while respecting invalidation and failure contracts.

Module project

Port the deterministic grid simulation to C++20, with a command-line build, automated tests, warnings enabled and an explicit account of representation differences from Python.

Core references

R11 C++ Core Guidelines editors — C++ Core Guidelines

R12 Kitware — CMake tutorial

R15 MIT OpenCourseWare — 6.004 Computation Structures

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 11.1 • MODULE 11

Compilation, linking and build systems

Learning goal Explain how source code becomes a runnable native program.

A C++ translation unit is compiled into object code; linking resolves references and combines the required pieces into a program or library. Headers usually provide declarations shared across translation units. A declaration describes an entity; a definition supplies its implementation or storage. Build errors therefore occur at different stages and require different evidence.

A build system records dependencies and compiler settings. Use a small CMake target rather than hardcoding paths throughout source files. Keep build products outside the source tree. Debug and optimized builds can differ substantially, so record which configuration produced a measurement. For this course, C++20 is a selected teaching baseline, not a claim that every feature is supported by every compiler.

WORKED EXAMPLE

A function can compile at a call site when its declaration is visible yet fail to link when no matching definition is supplied. A missing header usually fails earlier during compilation.

PRACTICE + PASS CRITERION

Build the included C++ starter, then split one function into a header and source file. Pass: reproduce and explain one compile error and one link error, then restore a warning-clean build.

ASSIGNED STUDY

R12: Getting Started and libraries; R11: Source Files.

CHECK YOUR UNDERSTANDING

1. What task does the linker perform?

2. Can valid declarations alone supply every function implementation?

3. Why record build configuration with a benchmark?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 11.2 • MODULE 11

Integer, floating-point and object representations

Learning goal Choose native types with explicit range and precision assumptions.

Native representations have finite range and precision. Do not assume int is a particular width in every environment. Fixed-width integer types are available when the implementation provides them; use them where a format or interface requires that width. Unsigned arithmetic wraps modulo its range, while signed integer overflow in ordinary C++ arithmetic is undefined behavior.

Floating-point operations approximate real arithmetic and have special values such as infinity and NaN on common IEEE-style implementations. Object layout can contain padding, so raw struct bytes are not a portable save or network format. Byte order, alignment and field encoding require explicit decisions. An apparently correct result on one machine does not establish portability.

WORKED EXAMPLE

An unsigned 8-bit value has 256 distinct values, 0 through 255. A 32-bit byte count can be too small for a large asset. Converting a negative signed value to an unsigned type may produce a very large number.

PRACTICE + PASS CRITERION

Print relevant sizes and numeric limits on the available compiler. Write checked conversions for a file length and tests for boundary values. Pass: do not trigger signed overflow as a test oracle; reject out-of-range input before converting.

ASSIGNED STUDY

R11: arithmetic and conversion guidance; R15: information encoding and arithmetic.

CHECK YOUR UNDERSTANDING

1. How many distinct bit patterns fit in eight bits?

2. Is ordinary signed integer overflow defined to wrap in C++?

3. Why is writing raw struct bytes not a portable format?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 11.3 • MODULE 11

Values, references, pointers and lifetimes

Learning goal Track where an object lives and how long access remains valid.

A value is an object's stored information. A reference aliases an existing object; a pointer stores an address-like value and can represent no object with nullptr. Neither a raw pointer nor a reference automatically owns what it refers to. Ownership determines who must release a resource; lifetime determines when the object exists.

Automatic storage duration often corresponds to stack allocation, but storage-duration rules are the reliable language concept. Returning a pointer or reference to a local object leaves a dangling access after that object's lifetime ends. Const expresses permitted modification through an interface, not universal thread safety or deep immutability. Draw ownership and lifetime diagrams before clever pointer code.

WORKED EXAMPLE

Passing const State& can avoid copying while allowing observation of a state owned by the caller. Returning a State by value can be safe and efficient; do not assume every value return causes an expensive extra copy.

PRACTICE + PASS CRITERION

Implement functions taking values, const references and pointers with a documented null policy. Pass: explain each lifetime and identify a deliberately supplied dangling-reference example without executing undefined behavior.

ASSIGNED STUDY

R11: Interfaces, Resource Management and lifetime safety guidance.

CHECK YOUR UNDERSTANDING

1. Does a raw pointer necessarily own its target?

2. Why is returning a reference to a local variable invalid?

3. Does const by itself make shared access thread-safe?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 11.4 • MODULE 11

Standard containers, iterators and error handling

Learning goal Use library abstractions while respecting invalidation and failure contracts.

Standard containers provide reusable storage and algorithms. std::vector offers contiguous elements and amortized growth behavior, but reallocation can invalidate pointers, references and iterators into its storage. Size is the number of live elements; capacity is storage reserved for possible growth. Reserving capacity does not construct that many elements.

Use standard algorithms when they make intent clearer. Check container preconditions and distinguish exceptional failure from ordinary absence. An optional value, status type or exception may each be appropriate under a documented policy. Do not catch every error and pretend success; preserve useful diagnostic context and avoid leaving partially updated state.

WORKED EXAMPLE

For a vector with size 3 and capacity 8, valid element indices are still 0, 1 and 2. Accessing index 7 is not allowed merely because storage has been reserved.

PRACTICE + PASS CRITERION

Port the unit list to std::vector and implement lookup with an explicit not-found result. Pass: test empty input, missing ID and growth; document which operations invalidate stored references.

ASSIGNED STUDY

R11: standard-library use and error handling; R12: testing integration.

CHECK YOUR UNDERSTANDING

1. Does reserve(100) make a vector's size 100?

2. Why can a pointer to a vector element become invalid after growth?

3. What is wrong with ignoring every exception?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 12

Ownership, abstractions and C++ design

Prerequisites Module 11.

Four lessons

12.1 RAII and resource ownership

Make cleanup follow object lifetime on success and failure paths.

12.2 Copy, move and exception safety

Define what transferring a resource means without duplicating ownership.

12.3 Interfaces, composition and polymorphism

Choose abstractions according to substitutability and measured cost.

12.4 Templates, concepts and data-oriented choices

Use generic code without hiding costs or exploding complexity.

Module project

Build a small resource manager with explicit ownership, safe move behavior, bounded interfaces, fault-injection tests and a written comparison of composition and inheritance.

Core references

R11 C++ Core Guidelines editors — C++ Core Guidelines

R12 Kitware — CMake tutorial

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 12.1 • MODULE 12

RAII and resource ownership

Learning goal Make cleanup follow object lifetime on success and failure paths.

Resource acquisition is initialization, or RAII, ties a resource to an owning object whose destructor releases it. The resource can be memory, a file, a mutex guard or a graphics handle. This makes cleanup occur on normal scope exit and during supported exception unwinding. Ownership should be visible in types rather than scattered comments.

Prefer unique ownership when one component controls lifetime. Shared ownership is useful for genuinely shared lifetimes but adds costs and can form cycles. Non-owning views must not outlive their owner. Wrapping an operating-system or GPU handle requires the correct release operation and context; a destructor is not magic if it runs after the device is gone.

WORKED EXAMPLE

A unique_ptr owns one object and releases it automatically. Passing a raw pointer to inspect that object can be fine when the caller guarantees the owner remains alive throughout the call.

PRACTICE + PASS CRITERION

Wrap a mock file/device resource whose open/close counters can be inspected. Inject failure after acquisition. Pass: each acquired resource is released exactly once and no view outlives its owner.

ASSIGNED STUDY

R11: Resource Management and smart-pointer ownership rules.

CHECK YOUR UNDERSTANDING

1. What does RAII connect?

2. Can shared ownership create a leak through cycles?

3. Why must device-resource destruction order be planned?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 12.2 • MODULE 12

Copy, move and exception safety

Learning goal Define what transferring a resource means without duplicating ownership.

Copying creates an independent value according to a type's contract. Moving transfers or reuses resources where appropriate; it is not merely a guaranteed byte copy. A moved-from standard-library object remains valid but may have an unspecified value unless its specific contract says more. std::move is a cast enabling move-aware overload resolution, not an operation that itself moves bytes.

Exception safety asks what remains true if an operation fails. A basic guarantee preserves validity and prevents leaks; a strong guarantee leaves observable state unchanged on failure. Build a replacement first and commit only after success when that guarantee is important. Prefer the rule of zero: compose well-behaved members rather than writing custom special-member functions unnecessarily.

WORKED EXAMPLE

To replace a loaded asset, validate a new asset before swapping it into the live slot. A failed parse then leaves the old working asset available instead of an empty or half-filled replacement.

PRACTICE + PASS CRITERION

Implement a move-only mock resource and transactional asset replacement. Pass: copying is prohibited, moving does not double-release, and injected load failure preserves the old asset.

ASSIGNED STUDY

R11: class special members, rule of zero and exception safety.

CHECK YOUR UNDERSTANDING

1. Does std::move itself guarantee that an object is moved?

2. What is the strong exception guarantee?

3. Why prefer the rule of zero?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 12.3 • MODULE 12

Interfaces, composition and polymorphism

Learning goal Choose abstractions according to substitutability and measured cost.

An interface should expose a coherent capability with a clear contract. Runtime polymorphism chooses behavior through a dynamic type, often via virtual functions. Static polymorphism resolves behavior through compile-time types. Neither is always faster or better; ownership, extensibility, binary boundaries and workload matter.

Composition keeps independent capabilities separate. An inheritance hierarchy is appropriate when derived objects can honor the base contract. A base class deleted polymorphically normally needs a virtual destructor. Avoid embedding every engine subsystem in one universal object. Small explicit interfaces make tests, replacement implementations and dependency analysis easier.

WORKED EXAMPLE

A Renderer interface might accept a scene description and produce a frame. A test renderer can record calls without drawing. It should not need to own gameplay rules or the network session.

PRACTICE + PASS CRITERION

Create interchangeable console and recording renderers behind one small interface. Pass: both satisfy the same contract, tests run without display output, and ownership/destructor behavior is explicit.

ASSIGNED STUDY

R11: Interfaces, Class Hierarchies and composition guidance.

CHECK YOUR UNDERSTANDING

1. What does substitutability require?

2. Why can a polymorphic base need a virtual destructor?

3. Why not put every responsibility into one engine class?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 12.4 • MODULE 12

Templates, concepts and data-oriented choices

Learning goal Use generic code without hiding costs or exploding complexity.

Templates express algorithms and types parameterized by compile-time information. Concepts constrain acceptable operations and improve interface meaning. Generic code still needs tests for relevant instantiations and clear compilation errors. Excessive instantiation can increase build times and binary size, so abstraction costs include developer time as well as runtime.

Data-oriented design starts from access patterns: which fields are processed together, how often, and in what order? Array-of-structures and structure-of-arrays layouts trade convenience, locality and vectorization opportunities. Do not adopt either as doctrine. Measure a representative loop and preserve correctness while comparing designs.

WORKED EXAMPLE

If a movement pass reads only positions and velocities, a structure-of-arrays layout can avoid fetching unrelated names and inventory fields. If each operation needs the complete object, that advantage may shrink or reverse.

PRACTICE + PASS CRITERION

Write a constrained generic clamp or interpolation function and benchmark two simple entity layouts. Pass: test contracts, record compiler/build mode and explain why the result is workload-specific.

ASSIGNED STUDY

R11: Templates and Generic Programming; R15: memory hierarchy motivation.

CHECK YOUR UNDERSTANDING

1. What does a concept constrain?

2. Is structure-of-arrays universally superior?

3. Name a cost of templates beyond runtime.

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 13

Data structures and their costs

Prerequisites Modules 06 and 11-12.

Four lessons

13.1 Complexity, arrays and linked structures

Predict asymptotic work while recognizing locality and constants.

13.2 Stacks, queues and ring buffers

Match ordering semantics to the intended workflow.

13.3 Hash tables and collision policies

Explain expected lookup behavior and its assumptions.

13.4 Trees, heaps and priority queues

Preserve structural invariants and choose the right query behavior.

Module project

Implement educational versions of a dynamic array, queue, hash table and heap, each with invariants and tests. Compare them to standard containers without replacing standard code in production merely for practice.

Core references

R13 MIT OpenCourseWare — 6.006 Introduction to Algorithms

R06 MIT OpenCourseWare — 6.042J Mathematics for Computer Science

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 13.1 • MODULE 13

Complexity, arrays and linked structures

Learning goal Predict asymptotic work while recognizing locality and constants.

Asymptotic notation describes how cost grows with input size under a stated model. Big-O is an upper bound, not an exact time or a synonym for worst case in every context. Arrays provide indexed access and contiguous storage. Linked structures connect separately located nodes and can support local insertion when the location is already known.

Searching for that location still costs work. Pointer chasing and allocation affect real performance beyond operation counts. A dynamic array can grow geometrically to achieve amortized constant-time append, even though some individual appends require linear relocation. Amortized analysis concerns a sequence of operations, not an average over random inputs.

WORKED EXAMPLE

Doubling capacity after each expansion makes total copied elements across n appends proportional to n. Growing capacity by one each time can copy approximately 1+2+...+(n-1), which is quadratic.

PRACTICE + PASS CRITERION

Implement a growable integer buffer for study. Count copied elements under doubling and one-at-a-time growth. Pass: derive the growth behavior and separately measure elapsed time without equating it to the proof.

ASSIGNED STUDY

R13: arrays, cost models and asymptotic analysis; R06: sums.

CHECK YOUR UNDERSTANDING

1. Does O(n) specify an exact runtime?

2. What is the total 1+...+(n-1) proportional to?

3. How does amortized analysis differ from random-input average-case analysis?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 13.2 • MODULE 13

Stacks, queues and ring buffers

Learning goal Match ordering semantics to the intended workflow.

A stack is last-in, first-out; a queue is first-in, first-out. Stacks support nested calls and undo-like history; queues support pending work and messages. A ring buffer reuses a fixed storage region by wrapping indices. It needs an unambiguous policy to distinguish empty from full.

Bounded structures force an overload decision: reject, overwrite, block or drop according to requirements. Silently overwriting input commands may be unacceptable even if overwriting old diagnostic samples is permitted. A single-threaded ring buffer is not automatically safe for concurrent producers and consumers; synchronization is a separate lesson.

WORKED EXAMPLE

Enqueue A, B and C into a FIFO queue, then dequeue twice: A and B leave first. Push the same values onto a stack, then pop twice: C and B leave first.

PRACTICE + PASS CRITERION

Implement a bounded queue with explicit full/empty results and wraparound tests. Pass: preserve FIFO order through more than three times its capacity in operations and never read an uninitialized slot.

ASSIGNED STUDY

R13: elementary data structures; R16: later concurrency comparison.

CHECK YOUR UNDERSTANDING

1. Which item leaves first after pushing A then B onto a stack?

2. Why does a ring buffer need a full/empty convention?

3. Does a correct single-threaded queue imply a thread-safe queue?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 13.3 • MODULE 13

Hash tables and collision policies

Learning goal Explain expected lookup behavior and its assumptions.

A hash table maps keys to buckets using a hash function. Different keys can share a bucket; collision handling is therefore required, not optional. Equality decides whether keys are actually the same. Load factor measures occupancy and affects performance according to the collision strategy.

Expected constant-time lookup relies on assumptions about hashing, load and inputs. Worst-case behavior can be much worse. Rehashing can invalidate references according to a container's contract. For untrusted inputs, adversarial collisions may matter. Hash values are not unique identifiers, encryption or proof that two objects are equal.

WORKED EXAMPLE

Keys 12 and 22 both map to bucket 2 under key modulo 10. A correct table retains both and compares actual keys during lookup.

PRACTICE + PASS CRITERION

Implement chaining or open addressing with collision tests and resizing. Pass: deliberately force many collisions, test missing keys and verify that distinct equal-hash keys remain distinct.

ASSIGNED STUDY

R13: hashing and dictionary data structures.

CHECK YOUR UNDERSTANDING

1. Does a matching hash prove two keys are equal?

2. What is the load factor for 30 entries in 10 buckets?

3. Why is expected O(1) not an unconditional guarantee?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 13.4 • MODULE 13

Trees, heaps and priority queues

Learning goal Preserve structural invariants and choose the right query behavior.

A binary search tree orders keys so that search can follow comparisons down a path. Its height determines many operation costs; an unbalanced tree can degrade to a chain. A heap maintains a different invariant: each parent has priority no worse than its children. It efficiently exposes an extreme element but does not globally sort all positions.

Priority queues are central to scheduling and pathfinding. Updating priorities requires a supported policy, such as handles, indexed heaps or inserting a new entry and ignoring stale ones. State the ordering and tie-break rules so tests and replays behave consistently.

WORKED EXAMPLE

A min-heap containing 2, 5 and 9 must expose 2 at the root, but its internal array is not promised to be a sorted sequence. A sorted insertion stream can make a naive search tree height n.

PRACTICE + PASS CRITERION

Implement a binary min-heap with push/pop and randomized comparison to a sorted reference. Pass: validate the heap invariant after every operation and test duplicate priorities.

ASSIGNED STUDY

R13: binary search trees, balanced trees and heaps.

CHECK YOUR UNDERSTANDING

1. Does a heap provide a fully sorted array?

2. What can the height of an unbalanced n-node search tree become?

3. Why define a tie-break rule in a priority queue?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 14

Algorithms for maps, simulation and tools

Prerequisites Modules 06, 09 and 13.

Four lessons

14.1 Searching, sorting and algorithm selection

Choose an algorithm from its preconditions and the data distribution.

14.2 Graph traversal and dependency ordering

Use graph algorithms with explicit edge semantics.

14.3 Dijkstra, A* and admissible heuristics

Build a correct pathfinder before optimizing its queue.

14.4 Dynamic programming, complexity limits and approximation

Recognize repeated subproblems and justify a practical solution.

Module project

Build a pathfinding visualizer and asset-dependency planner. Provide correctness arguments, unreachable cases, complexity analysis and measured comparisons on multiple map families.

Core references

R13 MIT OpenCourseWare — 6.006 Introduction to Algorithms

R06 MIT OpenCourseWare — 6.042J Mathematics for Computer Science

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 14.1 • MODULE 14

Searching, sorting and algorithm selection

Learning goal Choose an algorithm from its preconditions and the data distribution.

Binary search discards part of an ordered range after each comparison. Its invariant must preserve where a possible answer can remain. Sorting establishes order but incurs a cost; whether sorting first is worthwhile depends on how many later queries will reuse it. Stable sorting preserves relative order of equivalent keys.

Comparison sorting has well-known lower bounds under its model, while specialized methods can exploit bounded integer keys or other structure. Algorithm selection also considers memory, input size, nearly sorted data and worst-case behavior. Never apply a performance claim without its assumptions.

WORKED EXAMPLE

For 1024 ordered elements, binary search takes roughly ten halving decisions to narrow to one candidate, with exact comparisons depending on the implementation. An unsorted list does not satisfy the required invariant.

PRACTICE + PASS CRITERION

Implement binary search with a documented interval convention and test empty, one-element, duplicate and missing cases. Compare two sorts on random and nearly sorted inputs. Pass: verify output order and element preservation.

ASSIGNED STUDY

R13: binary search, sorting and comparison models.

CHECK YOUR UNDERSTANDING

1. What precondition does ordinary binary search require?

2. What does stable sorting preserve?

3. Why test element preservation as well as sortedness?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 14.2 • MODULE 14

Graph traversal and dependency ordering

Learning goal Use graph algorithms with explicit edge semantics.

Breadth-first search explores an unweighted graph by distance in edges; depth-first search follows a path before backtracking. Both need visited-state handling to avoid repeated traversal in cyclic graphs. A topological ordering exists only for a directed acyclic graph and places prerequisites before their dependents under the chosen edge convention.

Graph representation affects cost. Adjacency lists suit many sparse graphs; adjacency matrices can simplify dense or constant-time edge-existence queries at greater storage cost. A navigation graph is a model of movement, not the world itself: missing edges, one-way passages and dynamic obstacles change the problem.

WORKED EXAMPLE

With edges A->B and B->C meaning "must precede," A,B,C is a valid topological order. Adding C->A creates a cycle, so no such order exists.

PRACTICE + PASS CRITERION

Implement BFS and topological sorting. Pass: recover a shortest unweighted path, report unreachable goals, reject a cyclic dependency graph and justify the O(V+E) traversal bound for adjacency lists.

ASSIGNED STUDY

R13: graph representations, BFS, DFS and topological sorting.

CHECK YOUR UNDERSTANDING

1. When does BFS find a shortest path by edge count?

2. Can a directed graph with a cycle have a topological order?

3. Why mark visited nodes?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 14.3 • MODULE 14

Dijkstra, A* and admissible heuristics

Learning goal Build a correct pathfinder before optimizing its queue.

Dijkstra's algorithm finds shortest paths with nonnegative edge costs. A* adds a heuristic estimate of remaining cost, prioritizing f=g+h. An admissible heuristic never overestimates the true remaining cost. A consistent heuristic also respects a triangle-like condition across edges.

Implementation details matter: closed-set policies, stale queue entries and reopening nodes affect correctness when a heuristic is not consistent. Negative edge costs invalidate ordinary Dijkstra assumptions. For a four-neighbor unit-cost grid, Manhattan distance is a useful lower bound; diagonal motion or terrain costs require reconsidering the model. Faster search that returns an invalid route is not an improvement.

WORKED EXAMPLE

From (1,2) to (4,6), Manhattan distance is |4-1|+|6-2|=7 on a four-neighbor unit-cost grid. Allowing inexpensive diagonals changes the appropriate estimate.

PRACTICE + PASS CRITERION

Implement Dijkstra and A* against the same grid model. Pass: compare path costs on many small maps, include no-path cases and demonstrate a deliberately overestimating heuristic that can sacrifice optimality.

ASSIGNED STUDY

R13: shortest paths; R20: game navigation context.

CHECK YOUR UNDERSTANDING

1. What edge-cost condition does ordinary Dijkstra require?

2. What is Manhattan distance from (0,0) to (3,4)?

3. Why can inconsistent heuristics require reopening nodes?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 14.4 • MODULE 14

Dynamic programming, complexity limits and approximation

Learning goal Recognize repeated subproblems and justify a practical solution.

Dynamic programming stores solutions to overlapping subproblems under a recurrence. The difficult step is defining sufficient state and a valid dependency order. Memoization evaluates on demand; tabulation fills an ordered table. More dimensions can make a state space explode even when each transition is simple.

Complexity classes describe families of problems and reductions under formal models. NP-complete does not mean impossible on every instance, and a polynomial algorithm is not automatically fast enough in practice. Approximation, heuristics and bounded search can be appropriate when their guarantees and limits are explicit. Do not label an algorithm optimal merely because it found a good result once.

WORKED EXAMPLE

A budgeted loadout problem can use state (item_prefix, remaining_budget). Remembering only remaining_budget loses which items remain available and can produce an invalid recurrence.

PRACTICE + PASS CRITERION

Solve a small 0/1 knapsack instance by exhaustive search and dynamic programming. Pass: compare answers, derive state count, and describe how a heuristic would be evaluated without claiming a guarantee it lacks.

ASSIGNED STUDY

R13: dynamic programming; R06: reductions and asymptotic reasoning.

CHECK YOUR UNDERSTANDING

1. What two features motivate dynamic programming?

2. Does NP-complete mean no instance can be solved quickly?

3. Why keep a brute-force solver for tiny inputs?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 15

Programming languages, compilers and computability

Prerequisites Modules 11-14 and discrete proofs.

Four lessons

15.1 Language paradigms and type systems

Separate language semantics from syntax and implementation habits.

15.2 Lexing, parsing and syntax trees

Convert source text into a structured representation with diagnostics.

15.3 Bytecode, virtual machines and bounded execution

Implement a small execution model with explicit limits.

15.4 Automata, computability and compiler optimization

Know what analysis can establish and when transformation is legal.

Module project

Complete Gate C: a tested C++ simulation plus a tiny expression language with lexer, parser, evaluator, bytecode execution and resource limits.

Core references

R14 LLVM Project — Kaleidoscope and LLVM tutorials

R06 MIT OpenCourseWare — 6.042J Mathematics for Computer Science

R11 C++ Core Guidelines editors — C++ Core Guidelines

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 15.1 • MODULE 15

Language paradigms and type systems

Learning goal Separate language semantics from syntax and implementation habits.

Imperative code describes updates; functional style emphasizes expression evaluation and controlled side effects; object-oriented design organizes behavior through objects and interfaces. Real languages combine paradigms. A type system rejects certain invalid combinations according to its rules, but no ordinary type system proves all game requirements.

Static versus dynamic typing concerns when and how types are checked, not a simple ranking of language quality. Ownership-aware languages offer useful comparisons for systems design, while C remains important at binary interfaces and embedded boundaries. Learn enough C syntax and layout rules to read those interfaces, then preserve explicit ownership when calling them from C++.

WORKED EXAMPLE

A pure damage calculation can be tested independently of an imperative combat loop. The two styles cooperate: the loop manages time and effects, while the function computes a value.

PRACTICE + PASS CRITERION

Write one rule in imperative and pure-functional styles, then define a small typed C-compatible interface for a mock subsystem. Pass: state allocation ownership and reject type-invalid expressions in a toy checker.

ASSIGNED STUDY

R14: language frontend concepts; R11: interfaces and C-style boundary guidance.

CHECK YOUR UNDERSTANDING

1. Does static typing prove all logical requirements?

2. What advantage does a pure calculation offer tests?

3. What must a C-compatible ownership boundary specify?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 15.2 • MODULE 15

Lexing, parsing and syntax trees

Learning goal Convert source text into a structured representation with diagnostics.

A lexer groups characters into tokens such as numbers and operators. A parser applies a grammar to produce a syntax tree. Precedence and associativity determine structure: multiplication normally binds more tightly than addition, and subtraction usually groups left-to-right. An evaluator should operate on that structure rather than repeatedly guessing meaning from raw text.

Parsing failures need locations and useful messages. An input can be syntactically valid but semantically invalid, such as referring to an unknown variable. Keep these phases separate enough to test. Never implement a game-data parser by unrestricted execution of input text.

WORKED EXAMPLE

For 2+3*4, the tree has addition at the root and multiplication on the right, yielding 14. Parsing it as (2+3)*4 would yield 20 and violate the chosen precedence.

PRACTICE + PASS CRITERION

Implement tokens, recursive-descent parsing and evaluation for numbers, parentheses and four arithmetic operators. Pass: test precedence, associativity, malformed input, unknown names and division-by-zero policy.

ASSIGNED STUDY

R14: Kaleidoscope lexer, parser and abstract syntax tree chapters.

CHECK YOUR UNDERSTANDING

1. What separates lexing from parsing?

2. Under ordinary precedence, what is 2+3*4?

3. Why preserve source locations in a syntax tree?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 15.3 • MODULE 15

Bytecode, virtual machines and bounded execution

Learning goal Implement a small execution model with explicit limits.

A virtual machine executes a defined instruction set in software. A stack machine can use instructions such as push, add and multiply. Compiling an expression tree to bytecode separates source syntax from execution. A VM still needs a specification: stack effects, value types, branch behavior and error conditions.

Scripts and mods must be treated as potentially faulty input. Bound instruction count, memory use and access to host capabilities. A VM is not automatically a secure sandbox; unsafe host functions can defeat isolation. Keep the educational VM offline and limited, and do not equate a few checks with production security review.

WORKED EXAMPLE

The expression 2+3*4 can become PUSH 2, PUSH 3, PUSH 4, MUL, ADD. MUL consumes two values and pushes one, so the stack height drops by one.

PRACTICE + PASS CRITERION

Compile the expression language into stack bytecode and compare results to the tree evaluator. Pass: reject stack underflow, invalid opcodes and over-budget execution; preserve source context for errors.

ASSIGNED STUDY

R14: code generation and execution concepts; course security requirements.

CHECK YOUR UNDERSTANDING

1. What is the stack-height change of a binary add that pushes one result?

2. Why compare VM output to the tree evaluator?

3. Does using bytecode automatically create a secure sandbox?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 15.4 • MODULE 15

Automata, computability and compiler optimization

Learning goal Know what analysis can establish and when transformation is legal.

Finite automata recognize patterns using bounded state. More expressive computation models explain parsing and general programs. Some questions about arbitrary programs, including universal termination, are undecidable. This does not prevent proving termination for restricted programs or enforcing execution budgets.

Compiler optimization must preserve the language's observable semantics under its assumptions. Constant folding, dead-code elimination and inlining are examples, but floating-point reassociation can change results and undefined behavior can invalidate a programmer's intuition. Inspect generated code to learn, not to assume one assembly listing predicts every target. Correctness precedes instruction-count reduction.

WORKED EXAMPLE

Replacing 2+3 by 5 is straightforward for the defined integer literals. Reassociating (a+b)+c into a+(b+c) can change finite-precision floating-point results when magnitudes differ greatly.

PRACTICE + PASS CRITERION

Add constant folding to the toy compiler and test it against the unoptimized evaluator. Pass: include a finite-state recognizer, explain a termination restriction and document transformations deliberately excluded for semantic reasons.

ASSIGNED STUDY

R14: optimizer support; R06: computation models and proof limitations.

CHECK YOUR UNDERSTANDING

1. Does undecidability mean no individual program can be proven to terminate?

2. Why can floating-point reassociation change a result?

3. What is the primary correctness condition for an optimization?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.